Gestor de Turnos

Guía completa del proyecto: qué hace, cómo funciona por dentro y cómo explicarlo

---

Repositorio: github.com/riart5/gestor-turnos
Stack: React · Express · Prisma · SQLite · Vitest
Documento de referencia personal · Octubre 2026

Índice

1 El proyecto en 2 minutos (resumen para entrevistas)

2 El problema y la solución

3 Cómo se usa la app (recorrido por pantallas)

4 Tecnologías y por qué se eligieron

5 Arquitectura general

6 Estructura de carpetas, archivo por archivo

7 El modelo de datos (base de datos)

8 El backend: cómo viaja una petición

9 Autenticación y permisos (JWT + bcrypt)

10 Validación y manejo de errores

11 La API, endpoint por endpoint

12 El corazón: el servicio de intercambios

13 Notificaciones

14 El frontend: React por dentro

15 Las páginas y componentes

16 Estilos y diseño responsive

17 Datos de demo (seed)

18 Tests

19 Cómo arrancarlo y comandos útiles

20 Limitaciones conocidas y mejoras futuras

21 Preguntas típicas de entrevista (con respuesta)

22 Glosario

1. El proyecto en 2 minutos

Si solo puedes leer una página antes de una entrevista, que sea esta.

> **Cómo presentarlo en voz alta (unos 60 segundos)**
>
> «Es una aplicación web para gestionar los turnos de una planta de hospital. Surgió porque en sanidad los turnos se organizan con papeles y grupos de WhatsApp, y se pierde información, sobre todo cuando dos compañeros se cambian un turno.
>
> Hay dos roles: el sanitario, que ve su calendario y puede proponer cambios a compañeros, y el supervisor de planta, que gestiona el cuadrante y aprueba los cambios. Un intercambio pasa por tres pasos: lo propone un sanitario, lo acepta el compañero y lo aprueba el supervisor. Solo entonces se modifican los turnos, dentro de una transacción para que no quede nada a medias.
>
> El frontend está hecho con React y Vite, y el backend con Node, Express y Prisma sobre SQLite. La autenticación usa JWT, las contraseñas van cifradas con bcrypt y los datos de entrada se validan con Zod. La lógica de los intercambios tiene tests de integración con Vitest y Supertest.»

Los 5 puntos que más impresionan (y que debes dominar)

1. **Flujo de estados del intercambio.** Una solicitud pasa por `PENDING_PEER → PENDING_SUPERVISOR → APPROVED`, y puede terminar en `REJECTED` o `CANCELLED`. Es una pequeña *máquina de estados* (capítulo 12).
2. **Transacción al aprobar.** Los dos turnos cambian de dueño a la vez o no cambia ninguno (capítulo 12).
3. **Doble validación.** Los conflictos se comprueban al crear la propuesta y otra vez al aprobarla, porque entre medias el cuadrante puede haber cambiado (capítulo 12).
4. **Restricción en la base de datos.** `@@unique([userId, date])` garantiza que nadie tenga dos turnos el mismo día, aunque falle la validación del código (capítulo 7).
5. **El truco de la fecha temporal.** Permite intercambiar dos turnos del mismo día sin romper la restricción anterior (capítulo 12).

Cifras del proyecto

Concepto	Valor
Tablas en la base de datos	5 (Unit, User, Shift, SwapRequest, Notification)
Endpoints de la API	19
Pantallas	5 (Login, Mis turnos, Cuadrante, Intercambios, Avisos)
Tests automáticos	19 (todos pasan)
Datos de demo	1 supervisora, 6 sanitarios, unos 275 turnos

2. El problema y la solución

El problema real

En muchas plantas hospitalarias el cuadrante de turnos se gestiona con una hoja impresa colgada en el control de enfermería, un Excel que solo tiene la supervisora y grupos de WhatsApp. Esto provoca problemas concretos:

- **No hay una única versión de la verdad.** El papel dice una cosa, el Excel otra y el WhatsApp una tercera.
- **Los intercambios se pierden.** Dos compañeros acuerdan un cambio por mensaje y la supervisora no se entera, o se entera tarde.
- **Errores humanos.** Alguien acaba con dos turnos el mismo día, o un turno se queda sin cubrir.
- **Falta de trazabilidad.** No queda registro de quién pidió qué cambio y quién lo autorizó.

La solución

Problema	Cómo lo resuelve la app
Varias versiones del cuadrante	Un único cuadrante en la base de datos que todos consultan
Intercambios que se pierden	Flujo formal: propuesta → aceptación → aprobación, con estado visible
La supervisora no se entera	Notificaciones automáticas en cada paso
Dos turnos el mismo día	Validación en el código y restricción única en la base de datos
Turnos sin cubrir	Fila de «cobertura» en el cuadrante: marca en rojo los huecos
Sin trazabilidad	Cada solicitud guarda quién la creó, quién la resolvió y cuándo

Los dos roles

Sanitario (STAFF)	Supervisor/a (SUPERVISOR)
• Ve su calendario y su próximo turno • Consulta el cuadrante de su planta • Propone intercambios o cede turnos • Acepta o rechaza propuestas que recibe • Cancela sus propias propuestas • Recibe avisos	• Crea, cambia y elimina turnos • Aprueba o deniega intercambios • Ve todas las solicitudes de su planta • Ve la cobertura de cada turno • Da de alta personal en su planta • Recibe avisos

Tipos de turno

Se usan los tres turnos clásicos de planta hospitalaria:

Código	Nombre	Horario
M	Mañana	08:00 – 15:00

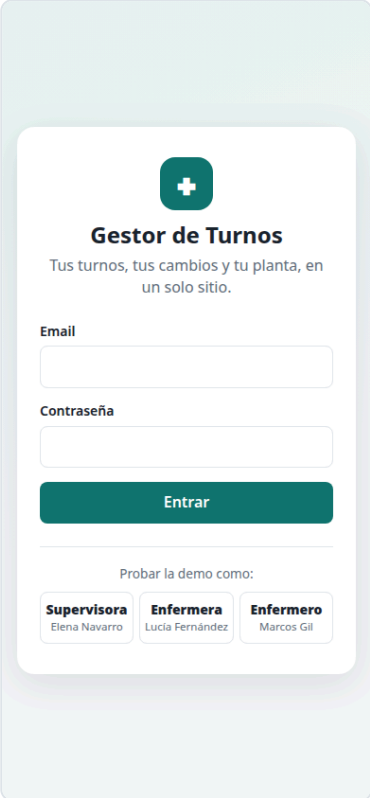
T	Tarde	15:00 – 22:00
N	Noche	22:00 – 08:00 (del día siguiente)

3. Cómo se usa la app

Pantalla de login

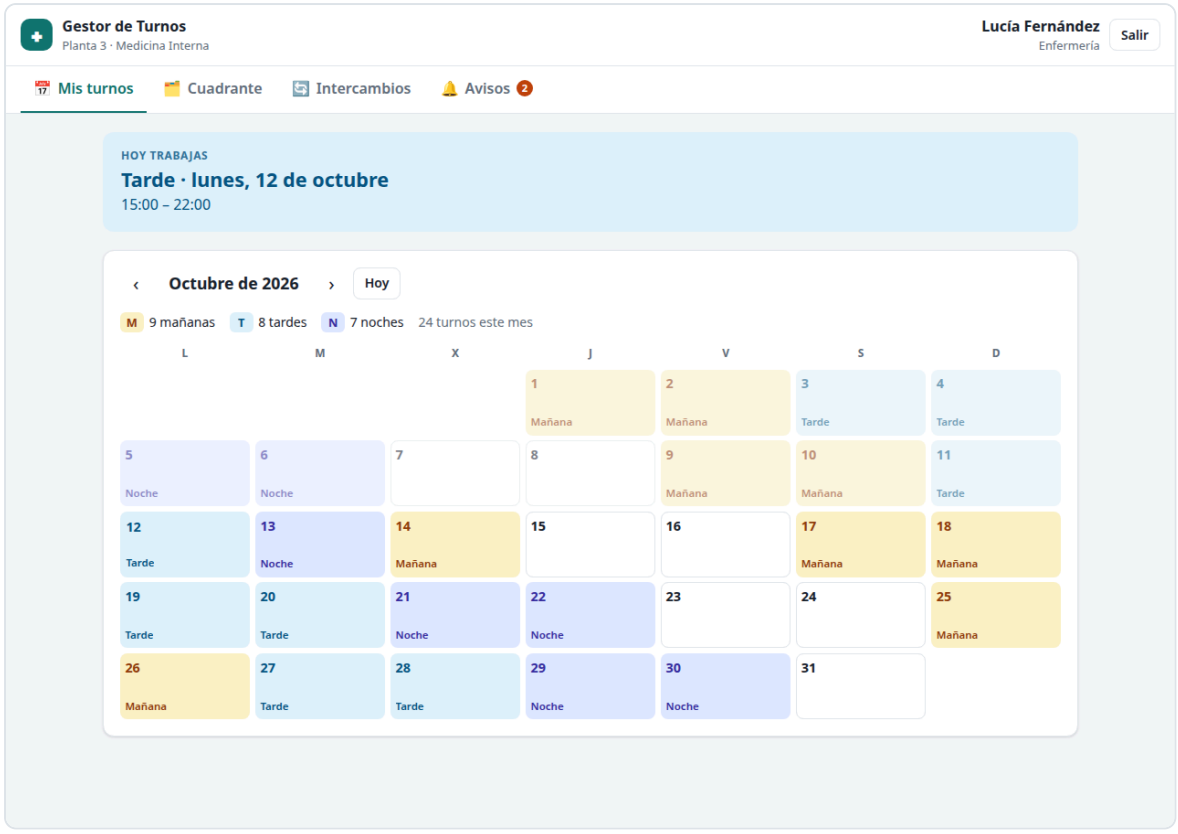
Formulario normal de email y contraseña. Debajo hay tres **botones de demo** que entran directamente como supervisora, como Lucía o como Marcos.

Están pensados para el portfolio: quien visite el proyecto puede probarlo en un clic sin registrarse.



Login en móvil

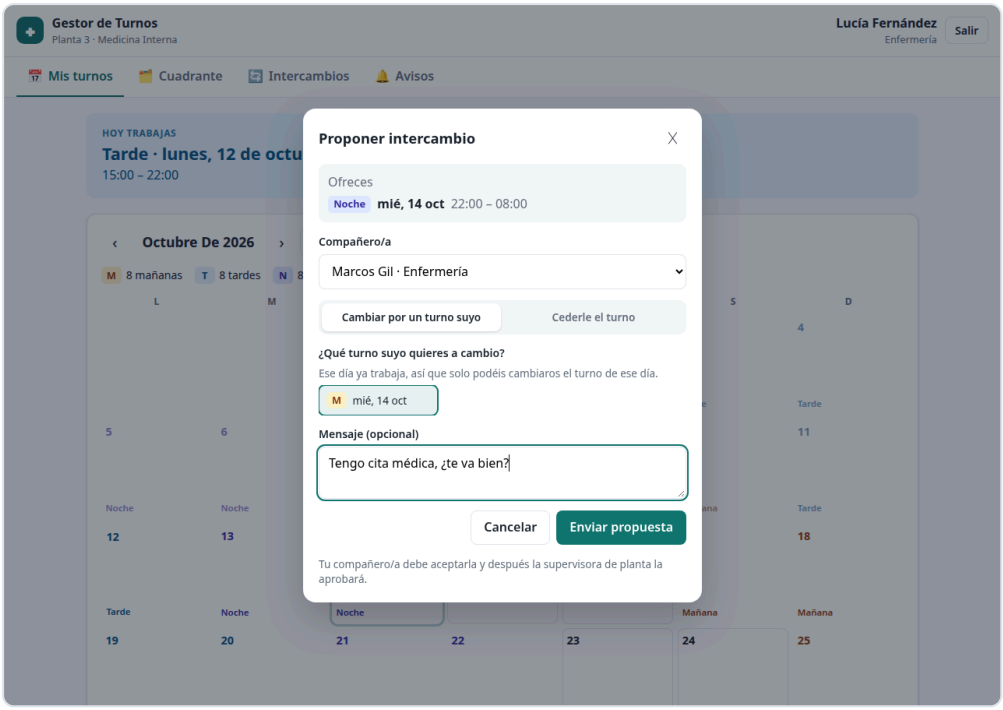
Mis turnos (calendario personal)



Calendario de Lucía: arriba el próximo turno; cada color es un tipo de turno.

- Arriba, una tarjeta con **el próximo turno** (o «Hoy trabajas»).
- Un resumen del mes: cuántas mañanas, tardes y noches.
- El calendario mensual, con un color por tipo de turno. Los días pasados aparecen atenuados.
- Un punto rojo indica que hay un intercambio en curso para ese turno.
- Al pulsar un día con turno futuro aparece el botón **«Proponer intercambio»**.

Proponer un intercambio



Lucía ofrece su noche del 14 a Marcos a cambio de la mañana de él ese mismo día.

Se elige al compañero y una de dos modalidades:

- **Cambiar por un turno suyo:** se muestran los turnos del compañero de los próximos 60 días que encajan (los días en los que tú no trabajas).
- **Cederle el turno:** se lo das sin recibir otro a cambio. No se puede si ese día él ya trabaja.

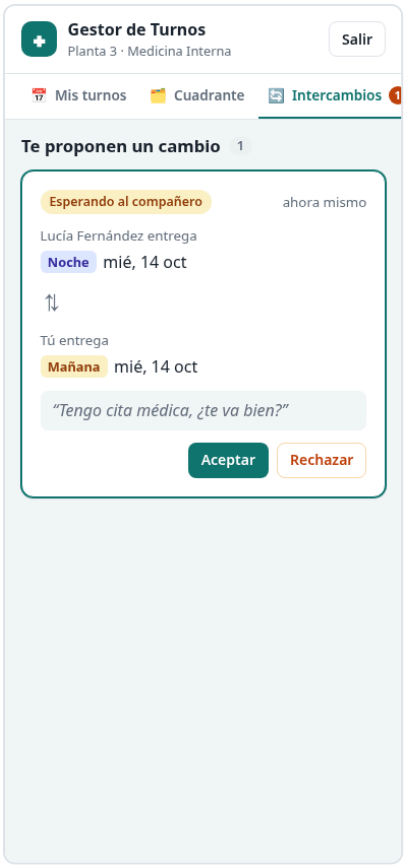
Si el compañero ya trabaja el día que ofreces, la app solo deja cambiar el turno de ese mismo día y lo explica con un aviso.

Intercambios: el compañero responde

La página de intercambios se divide en secciones según lo que necesita el usuario:

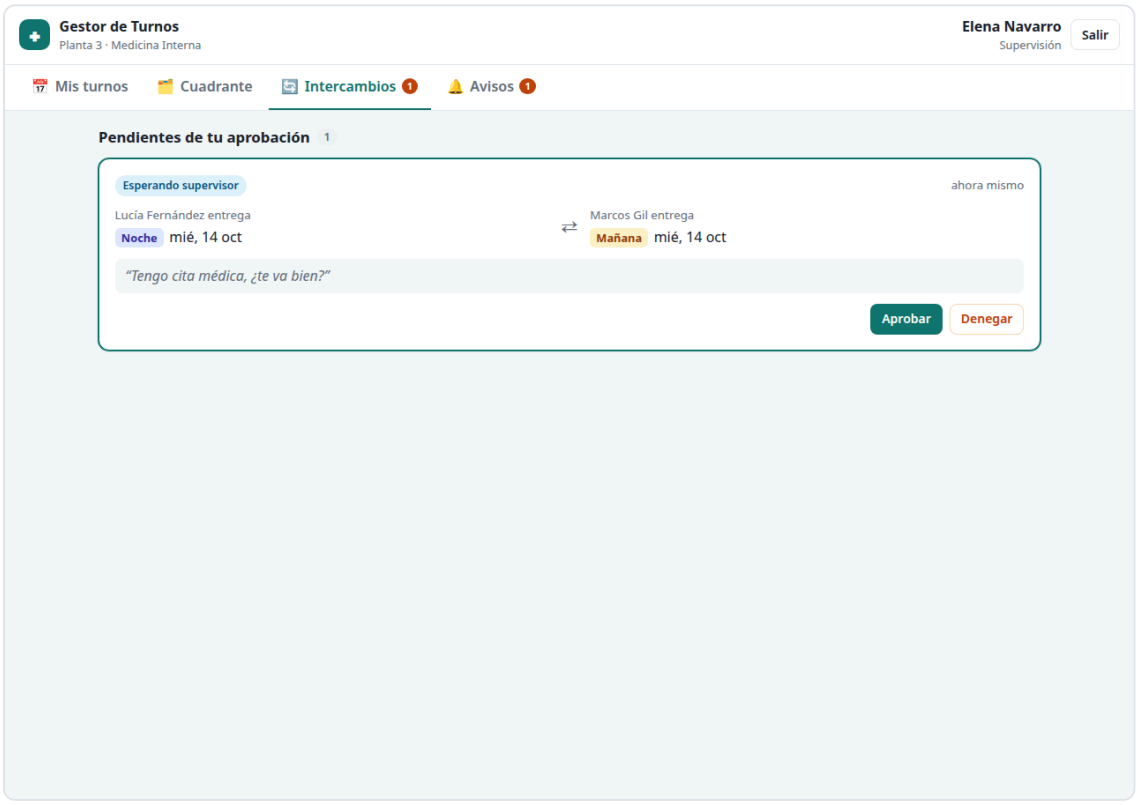
- **Te proponen un cambio:** lo que espera tu respuesta (resaltado), con los botones Aceptar y Rechazar.
- **En curso:** tus propuestas pendientes, con el botón Cancelar solicitud.
- **Historial:** las solicitudes ya cerradas.

La pestaña lleva un número rojo con las solicitudes que esperan tu acción.



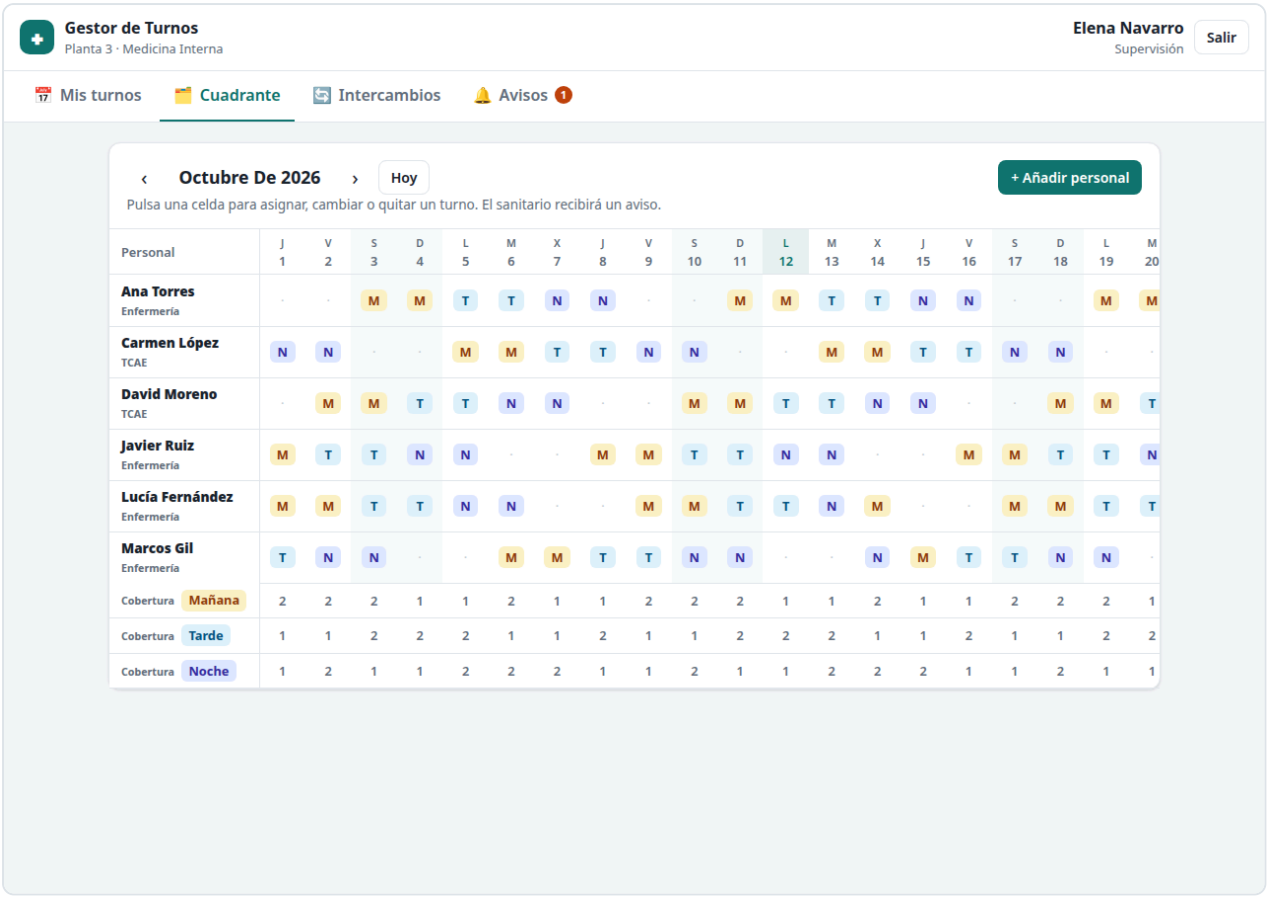
Marcos ve la propuesta en el móvil

La supervisora aprueba



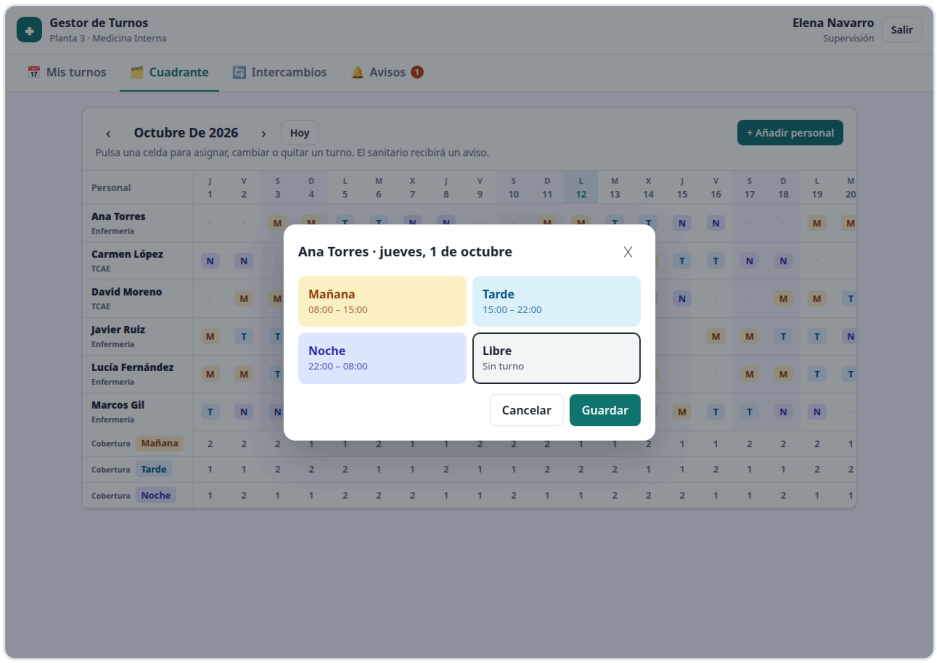
Vista de la supervisora: solicitudes pendientes de su aprobación.

Cuadrante de planta



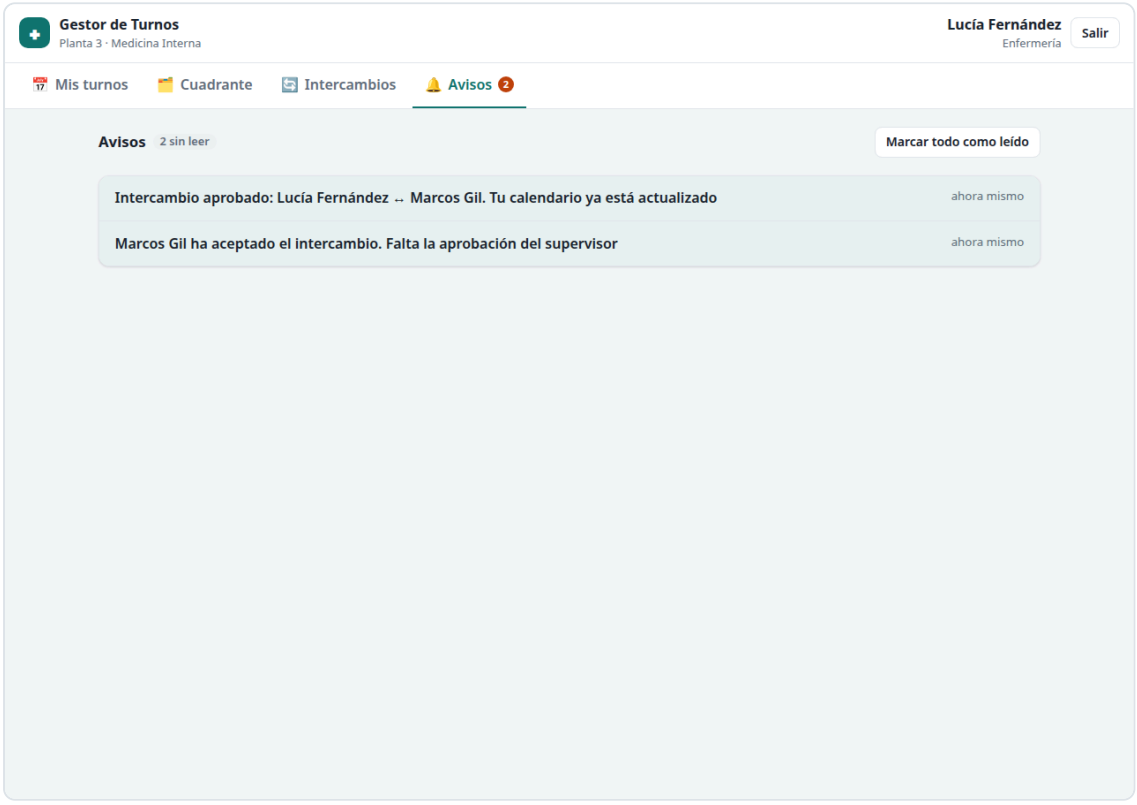
Cuadrante: una fila por persona y una columna por día. Abajo, la cobertura de cada turno.

- Lo ven todos, porque así un sanitario puede ver quién trabaja cuándo antes de proponer un cambio. Su propia fila sale resaltada.
- Solo la supervisora puede pulsar las celdas para asignar, cambiar o quitar un turno, y añadir notas.
- Las filas de **cobertura** cuentan cuánta gente hay en cada turno y cada día. Un 0 aparece en rojo.
- El botón **«+ Añadir personal»** permite dar de alta a nuevos sanitarios.



Editor de turno de la supervisora.

Avisos



Las notificaciones sin leer aparecen resaltadas.

4. Tecnologías y por qué se eligieron

Tecnología	Qué es	Por qué se eligió
React 19	Librería para construir interfaces con componentes	Es la más demandada en ofertas junior y encaja con una interfaz con mucho estado (calendario, modales, listas).
Vite	Herramienta de desarrollo y empaquetado	Arranca al instante y recarga en caliente. Es el estándar actual para crear proyectos React (sustituye a create-react-app).
React Router	Navegación entre páginas en el navegador	Permite tener URLs como `/cuadrante` sin recargar la página (SPA).
Node.js + Express 5	Servidor HTTP en JavaScript	Mismo lenguaje en frontend y backend. Express es simple y muy conocido. La versión 5 captura automáticamente los errores de las funciones `async`.
Prisma 6	ORM: permite trabajar con la base de datos usando objetos JavaScript en vez de SQL a mano	El esquema se define en un archivo legible (`schema.prisma`), genera consultas seguras y facilita cambiar de base de datos. Se usó la 6 porque la 7 y la 8 cambian mucho la configuración y tienen menos documentación.
SQLite	Base de datos en un único archivo	No hay que instalar nada (no había Docker). Con Prisma, pasar a PostgreSQL es cambiar el `provider`.
JWT (jsonwebtoken)	Token firmado que identifica al usuario	Autenticación sin sesiones en el servidor; fácil de usar desde una SPA.
bcryptjs	Función de hash para contraseñas	Nunca se guardan contraseñas en texto plano. bcrypt es lento a propósito, lo que dificulta los ataques de fuerza bruta.
Zod	Validación de datos con esquemas	Comprueba que lo que llega en cada petición tiene el formato correcto antes de usarlo.
cors	Middleware de Express	Permite que el frontend (otro puerto) llame a la API.
Vitest + Supertest	Framework de tests + cliente HTTP para tests	Vitest es rápido y moderno; Supertest permite lanzar peticiones reales a la app de Express sin arrancar el servidor.

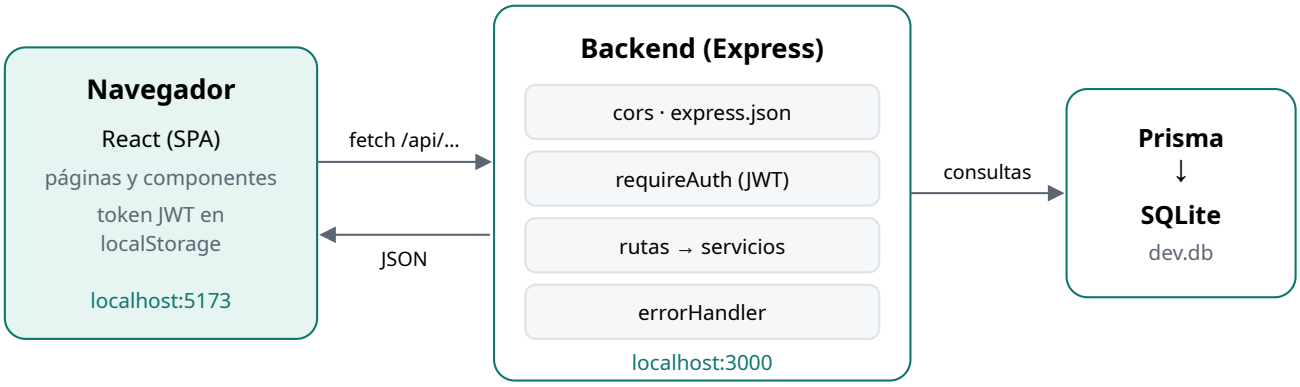
> **¿Por qué JavaScript y no TypeScript?**
> Fue una decisión consciente para centrarse primero en la lógica y en entender bien Express y React. TypeScript sería una buena mejora futura: detectaría errores de tipos (por ejemplo, pasar un `string` donde se espera un `id` numérico) antes de ejecutar.

¿Por qué CSS propio y no Tailwind o una librería de componentes?

Para demostrar que se domina CSS: variables, grid, flexbox, media queries y `position: sticky`. Además, la app no depende de ninguna librería visual.

5. Arquitectura general

Es una arquitectura **cliente-servidor** clásica con una **SPA** (Single Page Application) y una **API REST**:



El navegador habla con la API usando JSON; la API habla con la base de datos a través de Prisma.

Las capas del backend

El backend está organizado en capas. Cada capa tiene una responsabilidad:

Capa	Archivos	Responsabilidad
Rutas	`src/routes/*.js`	Recibir la petición HTTP, validar los datos con Zod, llamar a la lógica y devolver la respuesta.
Servicios	`src/services/swapService.js`	Lógica de negocio (reglas de los intercambios). No sabe nada de HTTP.
Middleware	`src/middleware/auth.js`	Funciones que se ejecutan antes de las rutas (comprobar el token, comprobar el rol).
Acceso a datos	`src/db.js` + Prisma	Hablar con la base de datos.
Utilidades	`src/lib/*.js`	Errores, tipos de turno, notificaciones, ocultar la contraseña.

> **Si te preguntan «¿por qué separaste la lógica de los intercambios en un servicio?»**
>
> «Porque es la parte más compleja y con más reglas. Separada de las rutas, cada función hace una cosa, se lee mejor y es fácil de testear. Las rutas quedan muy cortas: validan la entrada y llaman al servicio.»

¿Cómo se comunican frontend y backend en desarrollo?

El frontend corre en el puerto 5173 (Vite) y el backend en el 3000. En `vite.config.js` hay un **proxy**: cuando React hace `fetch('/api/shifts')`, Vite reenvía la petición a

`http://localhost:3000/api/shifts`. Así el frontend no necesita saber la URL del backend y se evitan problemas de CORS en desarrollo.

```
// frontend/vite.config.js
server: {
  proxy: { '/api': 'http://localhost:3000' },
}
```

6. Estructura de carpetas, archivo por archivo

```
gestor-turnos/
├── package.json              scripts para arrancar todo desde la raíz
├── README.md
├── backend/
│   ├── .env.example          plantilla de variables de entorno
│   ├── .env                  variables reales (NO se sube a git)
│   ├── package.json
│   ├── vitest.config.js      configuración de los tests
│   ├── prisma/
│   │   ├── schema.prisma     definición de las tablas
│   │   ├── seed.js           crea los datos de demo
│   │   └── dev.db            la base de datos SQLite (NO se sube)
│   ├── src/
│   │   ├── server.js         punto de entrada: arranca el servidor
│   │   ├── app.js            crea la app de Express y monta las rutas
│   │   ├── db.js             instancia única de PrismaClient
│   │   ├── middleware/
│   │   │   └── auth.js       firmar/verificar JWT, comprobar rol
│   │   ├── routes/
│   │   │   ├── auth.js       login y "quién soy"
│   │   │   ├── users.js      compañeros de planta y alta de personal
│   │   │   ├── shifts.js     CRUD de turnos
│   │   │   ├── swaps.js      endpoints de intercambios
│   │   │   └── notifications.js
│   │   ├── services/
│   │   │   └── swapService.js  TODA la lógica de intercambios
│   │   └── lib/
│   │       ├── errors.js     HttpError + manejador global de errores
│   │       ├── shiftTypes.js tipos M/T/N, fecha de hoy, descripciones
│   │       ├── notify.js     crear una notificación
│   │       └── publicUser.js quitar passwordHash de las respuestas
│   └── tests/
│       ├── globalSetup.js    prepara la base de datos de test
│       ├── helpers.js        utilidades: crear usuarios, turnos, peticiones
│       ├── auth.test.js      tests de login y permisos
│       └── swaps.test.js     tests del flujo de intercambios
└── frontend/
    ├── index.html
    ├── vite.config.js        plugin de React + proxy a la API
    └── src/
        ├── main.jsx          punto de entrada: monta React
        ├── App.jsx           definición de las rutas
        ├── styles.css        todos los estilos
        ├── lib/
        │   ├── api.js        cliente HTTP (fetch + token)
        │   ├── auth.jsx      contexto de autenticación
        │   ├── useFetch.js   hook para cargar datos
        │   ├── dates.js      utilidades de fechas
        │   └── shifts.js     tipos de turno y estados (para la UI)
        ├── components/
        │   ├── Layout.jsx    cabecera + pestañas + badges
        │   ├── Modal.jsx     ventana modal reutilizable
        │   ├── MonthNav.jsx  navegación entre meses
        │   ├── ShiftChip.jsx la etiqueta de color M/T/N
        │   └── SwapModal.jsx formulario para proponer intercambio
        └── pages/
            ├── LoginPage.jsx
```

```
            ├── CalendarPage.jsx      "Mis turnos"
            ├── RosterPage.jsx        "Cuadrante" (+ editor y alta)
            ├── SwapsPage.jsx         "Intercambios"
            └── NotificationsPage.jsx "Avisos"
```

¿Por qué `app.js` y `server.js` están separados?

`app.js` solo *crea* la aplicación (función `createApp()`), y `server.js` la *arranca* con `listen()`. Así los tests pueden crear la app y lanzarle peticiones con Supertest sin abrir un puerto real. Es un patrón muy habitual en Express.

Variables de entorno (`.env`)

Variable	Para qué sirve
`DATABASE_URL`	Dónde está la base de datos (`file:./dev.db`)
`JWT_SECRET`	Clave secreta para firmar los tokens. Si alguien la conoce, puede fabricar tokens falsos. Por eso nunca se sube a git.
`PORT`	Puerto del servidor (3000)
`CORS_ORIGIN`	Qué origen puede llamar a la API (el frontend)

Se cargan con la opción nativa de Node `--env-file=.env` (disponible desde Node 20), sin necesidad de instalar la librería `dotenv`.

7. El modelo de datos

La base de datos se define en `backend/prisma/schema.prisma`. Con `npx prisma db push`, Prisma crea las tablas en SQLite y genera el cliente JavaScript para consultarlas.

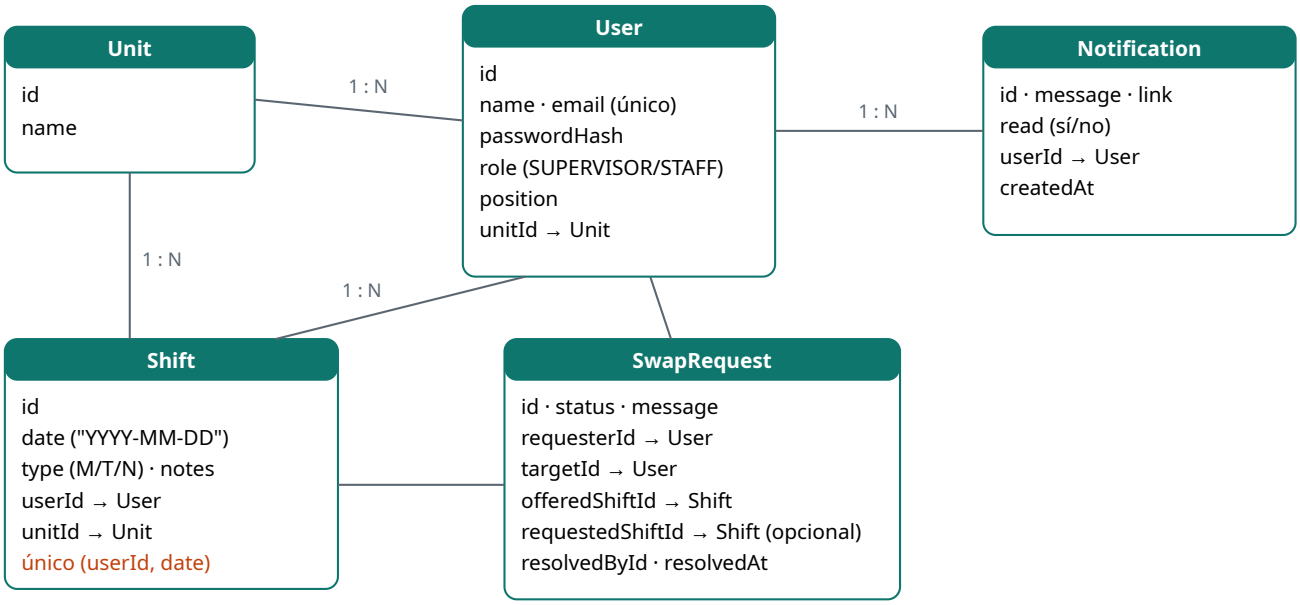


Diagrama entidad-relación simplificado.

Las tablas, una a una

Unit (planta)

Una planta del hospital, como «Planta 3 · Medicina Interna». Sirve para **aislar** los datos: cada usuario solo ve los turnos y compañeros de su planta. Aunque la demo tiene una sola planta, el modelo admite muchas.

User (usuario)

- `email` es **único**: no puede haber dos cuentas con el mismo email.
- `passwordHash`: guarda el *hash* de bcrypt, nunca la contraseña.
- `role`: `"SUPERVISOR"` o `"STAFF"`. Decide qué puede hacer.
- `position`: la categoría profesional (Enfermería, TCAE…). Es solo informativa.

Shift (turno)

```
model Shift {
  id     Int    @id @default(autoincrement())
  date   String // "YYYY-MM-DD" (día de inicio del turno)
  type   String // "M" mañana | "T" tarde | "N" noche
  notes  String?
  userId Int
  unitId Int
  ...
  @@unique([userId, date])   // un sanitario no puede tener dos turnos el mismo día
  @@index([unitId, date])    // acelera la consulta "turnos de mi planta entre dos fechas"
}
```

¿Por qué la fecha es un texto y no un DateTime?

Un turno pertenece a un **día**, no a un instante. Si se guardara como `DateTime`, entrarían en juego las zonas horarias: «12 de octubre a las 00:00 en España» son las 22:00 del día 11 en UTC, y un turno podría acabar mostrándose el día anterior. Con el formato `"YYYY-MM-DD"` eso no pasa. Además, este formato se ordena bien como texto: `"2026-10-05" < "2026-10-12"`. Por eso se pueden hacer consultas como `date: { gte: from, lte: to }`.

¿Por qué `role`, `type` y `status` son String y no enum?

Durante mucho tiempo Prisma no admitió `enum` con SQLite. Se usa `String` y los valores válidos se controlan en el código con Zod (por ejemplo, `z.enum(['M','T','N'])`). Si se pasa a PostgreSQL, se podrían convertir en enums reales.

La restricción `@@unique([userId, date])` es clave

Es una garantía **a nivel de base de datos**: aunque hubiera un error en el código, SQLite se negaría a guardar dos turnos de la misma persona el mismo día. Es la «última línea de defensa». Si ocurre, Prisma lanza el error `P2002`, que el manejador de errores convierte en un `409 Conflict`.

SwapRequest (solicitud de intercambio)

- `requester`: quien propone. `target`: el compañero que recibe la propuesta.
- `offeredShift`: el turno que entrega quien propone.
- `requestedShift`: el turno que recibe a cambio. Si es `null`, es una **cesión**.
- `status`: el estado de la solicitud (ver capítulo 12).
- `resolvedById` y `resolvedAt`: quién la cerró y cuándo (trazabilidad).

Como hay **dos relaciones** de SwapRequest con User (requester y target) y otras dos con Shift, Prisma necesita darles nombre (`@relation("SwapRequester")`…) para distinguirlas.

`onDelete: Cascade` en los turnos: si se borra un turno, se borran también las solicitudes que lo incluían.

Notification (aviso)

Un mensaje para un usuario, con un `link` opcional a la página relacionada y un campo `read`. Tiene un índice en `(userId, read)` porque la consulta más habitual es «avisos de este usuario».

8. El backend: cómo viaja una petición

Vamos a seguir una petición real de principio a fin: **Marcos acepta un intercambio** (`POST /api/swaps/7/accept`).

1. **React** llama a `api.post('/swaps/7/accept')` . El cliente de `api.js` añade la cabecera `Authorization: Bearer <token>` .
2. **Vite** reenvía la petición al puerto 3000.
3. **Express** la pasa por los middlewares globales: `cors()` (permite el origen) y `express.json()` (convierte el cuerpo JSON en `req.body`).
4. La ruta `/api/swaps` está montada con **`requireAuth`** : verifica el token, busca al usuario en la base de datos y lo guarda en `req.user` .
5. El router de intercambios encuentra `POST /:id/accept` y llama a `respondAsPeer(req.user, 7, true)` .
6. El **servicio** comprueba las reglas (¿la solicitud va dirigida a Marcos? ¿sigue pendiente? ¿hay conflictos?), actualiza el estado y crea los avisos.
7. La ruta responde con `res.json(...)` , con la solicitud actualizada.
8. Si algo falla en cualquier punto (por ejemplo, el servicio lanza `forbidden()`), el error salta directamente al **`errorHandler`** , que responde con el código HTTP y el mensaje adecuados.

backend/src/app.js

```
export function createApp() {
  const app = express();
  app.use(cors({ origin: process.env.CORS_ORIGIN ?? 'http://localhost:5173' }));
  app.use(express.json());

  app.get('/api/health', (req, res) => res.json({ ok: true }));
  app.use('/api/auth', authRoutes);                          // público (login)
  app.use('/api/users', requireAuth, userRoutes);            // todo lo demás exige login
  app.use('/api/shifts', requireAuth, shiftRoutes);
  app.use('/api/swaps', requireAuth, swapRoutes);
  app.use('/api/notifications', requireAuth, notificationRoutes);

  app.use('/api', (req, res, next) => next(notFound('Ruta no encontrada')));
  app.use(errorHandler);                                     // siempre el último
  return app;
}
```

¿Qué es un middleware?

Una función `(req, res, next)` que se ejecuta en medio del camino de la petición. Puede:

- modificar `req` (por ejemplo, añadir `req.user`),
- responder directamente (cortando la cadena),
- o llamar a `next()` para pasar al siguiente. Si se llama a `next(error)` , se salta directamente al manejador de errores.

El **orden importa**: `express.json()` debe ir antes que las rutas (si no, `req.body` estaría vacío) y `errorHandler` debe ir el último.

Express 5 y los errores en funciones async

Las rutas son `async` y lanzan errores con `throw`. En Express 4 había que envolver cada ruta en un `try/catch` o usar una librería, porque si no el error se perdía y la petición se quedaba colgada. **Express 5 captura automáticamente las promesas rechazadas** y las manda al `errorHandler`. Por eso las rutas son tan cortas:

```
router.post('/:id/accept', async (req, res) => {
  res.json(await respondAsPeer(req.user, swapId(req), true));
});
```

La conexión a la base de datos

backend/src/db.js

```
import { PrismaClient } from '@prisma/client';
export const prisma = new PrismaClient();
```

Se crea **una única instancia** y se reutiliza en todo el backend. Cada `PrismaClient` abre su propio grupo de conexiones, así que crear uno por petición sería un desperdicio.

9. Autenticación y permisos

Conviene distinguir dos conceptos:

- **Autenticación:** ¿quién eres? Se resuelve con el login y el JWT.
- **Autorización:** ¿qué puedes hacer? Se resuelve con los roles y las comprobaciones de propiedad.

Login paso a paso

`backend/src/routes/auth.js`

```
router.post('/login', async (req, res) => {
  const { email, password } = loginSchema.parse(req.body);           // 1. validar
  const user = await prisma.user.findUnique({ where: { email: email.toLowerCase() } });
  if (!user || !(await bcrypt.compare(password, user.passwordHash))) { // 2. comprobar
    throw new HttpError(401, 'Email o contraseña incorrectos');
  }
  res.json({ token: signToken(user), user: publicUser(user) });       // 3. emitir token
});
```

> **Detalle de seguridad: el mismo mensaje para los dos fallos**
> Tanto si el email no existe como si la contraseña es incorrecta, el mensaje es el mismo: «Email o contraseña incorrectos». Si fueran distintos, un atacante podría averiguar qué emails están registrados (esto se llama *enumeración de usuarios*).

bcrypt: cómo se guardan las contraseñas

Al crear un usuario: `bcrypt.hash('turnos123', 10)` produce algo como `$2b$10$N9qo8uLOickgx2ZMRZoMy...`. Este hash:

- es de **un solo sentido**: no se puede volver a la contraseña original;
- incluye una **sal** aleatoria: dos usuarios con la misma contraseña tienen hashes distintos;
- es **lento a propósito**: el 10 es el «coste» (2^{10} rondas), lo que hace carísimo probar millones de contraseñas.

Al hacer login, `bcrypt.compare()` vuelve a calcular el hash con la contraseña introducida y la sal guardada, y compara los resultados.

La función `publicUser()` elimina `passwordHash` antes de enviar un usuario al frontend, para que el hash nunca salga del servidor. Hay un test que lo comprueba.

JWT: el token de sesión

Un JWT (JSON Web Token) tiene tres partes separadas por puntos: `cabecera.contenido.firma`.

```
export function signToken(user) {
  return jwt.sign({ sub: user.id }, process.env.JWT_SECRET, { expiresIn: '7d' });
}
```

- El **contenido** (*payload*) lleva `sub` (el id del usuario) y la fecha de caducidad. Ojo: no está cifrado, solo codificado en Base64. Cualquiera puede leerlo, así que nunca se mete información sensible.
- La **firma** se calcula con `JWT_SECRET`. Si alguien modifica el contenido (por ejemplo, cambia `sub` por otro id), la firma ya no coincide y el token se rechaza.
- Caduca a los **7 días**.

El middleware `requireAuth`

```
export async function requireAuth(req, res, next) {
  const header = req.headers.authorization ?? '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return next(new HttpError(401, 'No autenticado'));
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);   // firma + caducidad
    const user = await prisma.user.findUnique({ where: { id: payload.sub } });
    if (!user) return next(new HttpError(401, 'Usuario no válido'));
    req.user = user;                                              // disponible en las rutas
    next();
  } catch {
    next(new HttpError(401, 'Sesión caducada o no válida'));
  }
}
```

Se busca al usuario en la base de datos en cada petición. Es una consulta extra, pero garantiza que si se borra un usuario o se le cambia el rol, el cambio tiene efecto inmediato y no a los 7 días.

Autorización: tres niveles

Nivel	Ejemplo	Cómo se comprueba
Por rol	Solo el supervisor crea turnos	Middleware `requireSupervisor` en la ruta
Por planta	Solo ves turnos de tu planta	Todas las consultas filtran por `unitId: req.user.unitId`
Por propiedad	Solo puedes ofrecer *tus* turnos; solo el destinatario acepta	Comparaciones en el servicio: `offeredShift.userId !== user.id`

> **Regla de oro: el frontend no protege nada**
> Aunque el frontend esconde el botón «Aprobar» a los sanitarios, la seguridad real está en el backend. Cualquiera podría abrir la consola del navegador y hacer `fetch('/api/swaps/7/approve', ...)` a mano; el backend lo rechazaría con un 403. Los tests verifican estas reglas directamente contra la API.

10. Validación y manejo de errores

Validación con Zod

Nunca hay que fiarse de lo que llega del cliente. Cada ruta define un **esquema** con la forma esperada de los datos:

```
const shiftSchema = z.object({
  userId: z.number().int(),
  date: z.string().regex(/^\d{4}-\d{2}-\d{2}$/, 'Formato de fecha YYYY-MM-DD'),
  type: z.enum(['M', 'T', 'N']),
  notes: z.string().trim().max(200).optional().nullable(),
});

const data = shiftSchema.parse(req.body);   // si no cumple, lanza ZodError → 400
```

Ventajas:

- Si los datos no son válidos, `parse` lanza un `ZodError` y la ruta no sigue ejecutándose.
- `data` solo contiene los campos definidos: si alguien envía un campo extra, como `role: "SUPERVISOR"`, se descarta. Esto evita la vulnerabilidad de *mass assignment*.
- Se reutilizan esquemas: `shiftSchema.partial()` hace todos los campos opcionales para el `PATCH`.
- Para los parámetros de la URL se usa `z.coerce.number()`, que convierte el texto `"5"` en el número `5`.

Errores HTTP propios

`backend/src/lib/errors.js`

```
export class HttpError extends Error {
  constructor(status, message) { super(message); this.status = status; }
}
export const badRequest = (msg) => new HttpError(400, msg);
export const forbidden = (msg = 'No tienes permiso para esta acción') => new HttpError(403,
msg);
export const notFound  = (msg = 'No encontrado') => new HttpError(404, msg);
export const conflict  = (msg) => new HttpError(409, msg);
```

Desde cualquier sitio (rutas o servicios) basta con `throw forbidden('...')`. El error sube hasta el manejador global.

El manejador global

```
export function errorHandler(err, req, res, next) {
  if (err instanceof HttpError) return res.status(err.status).json({ error: err.message });
  if (err instanceof ZodError)  return res.status(400).json({ error: 'Datos no válidos',
details: err.issues });
  if (err.code === 'P2002')     return res.status(409).json({ error: 'Ya existe un registro con
esos datos...' });
  console.error(err);
  res.status(500).json({ error: 'Error interno del servidor' });   // no se filtran detalles
internos
}
```

Express reconoce que es un manejador de errores porque recibe **4 parámetros**. Todas las respuestas de error tienen el mismo formato `{ error: "mensaje" }`, así que el frontend siempre sabe dónde leer el mensaje.

Códigos HTTP que usa la API

Código	Significado	Ejemplo en la app
200 OK	Todo bien	Listar turnos, aceptar un intercambio
201 Created	Se ha creado algo	Crear un turno o una solicitud
204 No Content	Bien, pero sin nada que devolver	Borrar un turno, marcar avisos como leídos
400 Bad Request	Datos incorrectos	Fecha mal escrita, turno pasado, compañero de otra planta
401 Unauthorized	No estás identificado	Sin token, token caducado, contraseña mal
403 Forbidden	Identificado, pero sin permiso	Un sanitario intenta crear un turno
404 Not Found	No existe	Solicitud o turno inexistente
409 Conflict	Choca con el estado actual	Dos turnos el mismo día, solicitud ya resuelta
500	Error inesperado del servidor	Un bug

11. La API, endpoint por endpoint

Todas las rutas empiezan por `/api`. Todas, salvo login y health, necesitan la cabecera `Authorization: Bearer <token>`.

Autenticación

Método y ruta	Quién	Qué hace
`POST /auth/login`	Público	Recibe `{email, password}` y devuelve `{token, user}`
`GET /auth/me`	Logueado	Devuelve el usuario actual con su planta. El frontend lo usa al cargar la página para saber si la sesión sigue activa.
`GET /health`	Público	Devuelve `{ok: true}`. Sirve para comprobar que el servidor está vivo.

Usuarios

Método y ruta	Quién	Qué hace
`GET /users`	Logueado	Lista los compañeros de tu planta (sin hash de contraseña)
`POST /users`	Supervisor	Da de alta un sanitario en la planta del supervisor. El rol siempre es STAFF.

Turnos

Método y ruta	Quién	Qué hace
`GET /shifts?from=&to=&userId=`	Logueado	Turnos de tu planta entre dos fechas. `userId` es opcional, para filtrar por persona.
`POST /shifts`	Supervisor	Crea un turno y avisa al sanitario
`PATCH /shifts/:id`	Supervisor	Modifica un turno (tipo, persona, notas…), cancela los intercambios pendientes que lo incluyen y avisa a los afectados
`DELETE /shifts/:id`	Supervisor	Elimina un turno y avisa

El cambio de un turno avisa de forma distinta según el caso: si cambia de persona, se avisa a la anterior («se te ha retirado») y a la nueva («se te ha asignado»); si solo cambia el tipo, se avisa del cambio.

Intercambios

Método y ruta	Quién	Qué hace
`GET /swaps`	Logueado	Un sanitario ve las suyas (enviadas y recibidas); el supervisor ve todas las de su planta
`POST /swaps`	Logueado	Crea una propuesta: `{offeredShiftId, targetId, requestedShiftId?, message?}`

`POST /swaps/:id/accept`	Destinatario	Acepta: pasa a PENDING_SUPERVISOR
`POST /swaps/:id/reject`	Destinatario	Rechaza: pasa a REJECTED
`POST /swaps/:id/cancel`	Quien la creó	Cancela: pasa a CANCELLED
`POST /swaps/:id/approve`	Supervisor	Aprueba y ejecuta el intercambio de turnos
`POST /swaps/:id/deny`	Supervisor	Deniega: pasa a REJECTED

> **¿Por qué `POST /swaps/:id/accept` y no `PATCH /swaps/:id {status: ...}`?**
> Porque cada acción tiene reglas distintas: quién puede hacerla, desde qué estado y qué efectos tiene. Con un endpoint por acción, cada regla está en su sitio y nadie puede saltar directamente de PENDING_PEER a APPROVED cambiando un campo. Es un patrón habitual para modelar *acciones* sobre un recurso.

Notificaciones

Método y ruta	Quién	Qué hace
`GET /notifications`	Logueado	Tus últimos 50 avisos, del más reciente al más antiguo
`POST /notifications/:id/read`	Logueado	Marca uno como leído (solo si es tuyo)
`POST /notifications/read-all`	Logueado	Marca todos como leídos

En `/:id/read` se usa `updateMany` con `where: { id, userId: req.user.id }`. Si el aviso no es tuyo, simplemente no se actualiza nada: así nadie puede tocar los avisos de otro.

Ejemplo de petición y respuesta

```
POST /api/swaps
Authorization: Bearer eyJhbGciOi...
Content-Type: application/json

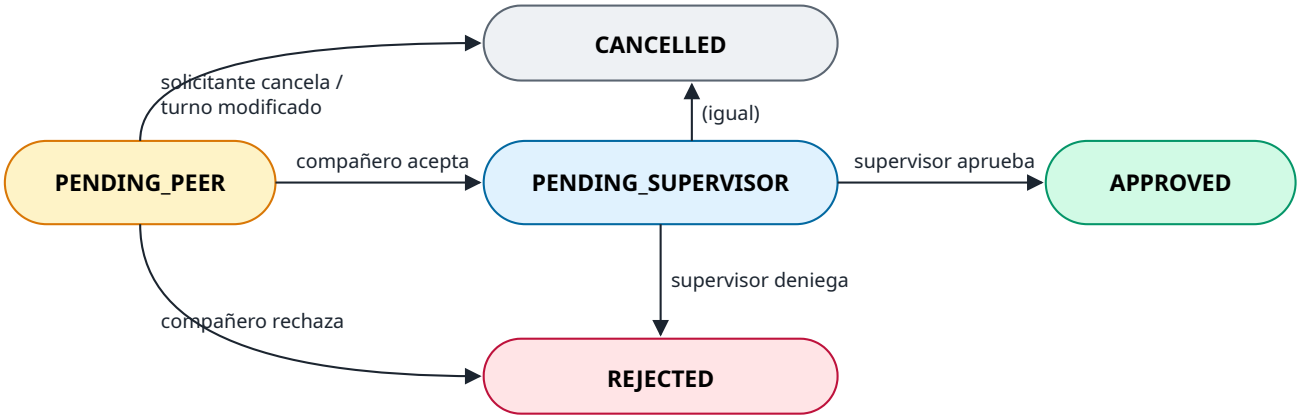
{ "offeredShiftId": 42, "targetId": 3, "requestedShiftId": 57, "message": "Tengo médico" }

→ 201 Created
{ "id": 7, "status": "PENDING_PEER", "requester": { "id": 2, "name": "Lucía Fernández", ... },
  "target": { "id": 3, "name": "Marcos Gil", ... },
  "offeredShift": { "id": 42, "date": "2026-10-14", "type": "N", ... },
  "requestedShift": { "id": 57, "date": "2026-10-14", "type": "M", ... }, ... }
```

12. El corazón: el servicio de intercambios

Archivo: `backend/src/services/swapService.js`. Es la parte más importante del proyecto y la que más probablemente te pidan explicar.

La máquina de estados



Estados posibles de una solicitud. APPROVED, REJECTED y CANCELLED son estados finales.

Cada función del servicio comprueba que la solicitud está en el estado correcto antes de actuar. Por ejemplo, el supervisor no puede aprobar algo que el compañero aún no ha aceptado (devuelve un 409).

Crear una solicitud: `createSwap`

Se comprueban, en este orden:

1. El turno ofrecido existe, es **tuyo** (si no, 403) y **no ha pasado** (si no, 400).
2. No te lo ofreces a **ti mismo**.
3. El compañero existe y es de **tu planta**.
4. Si pides un turno a cambio, ese turno existe, es **del compañero** y no ha pasado.
5. No hay ya otra solicitud **pendiente** sobre el mismo turno ofrecido (si no, 409).
6. **Conflictos**: después del cambio, nadie puede quedar con dos turnos el mismo día (`assertNoConflicts`).

Si todo va bien, se crea la solicitud en estado `PENDING_PEER` y se avisa al compañero con un mensaje como «Lucía Fernández quiere cambiar su Noche del 14/10/2026 por tu Mañana del 14/10/2026».

La comprobación de conflictos

```
async function assertNoConflicts({ requesterId, targetId, offeredShift, requestedShift }, db =
prisma) {
  // ¿El compañero ya trabaja el día del turno que va a recibir?
  // (sin contar el turno que él entrega, si es del mismo día)
  const targetBusy = await db.shift.findFirst({
    where: {
      userId: targetId,
      date: offeredShift.date,
      ...(requestedShift && { id: { not: requestedShift.id } }),
    },
  });
  if (targetBusy) throw conflict(`El compañero ya tiene un turno el ${offeredShift.date}`);

  // Lo mismo para quien propone, con el turno que va a recibir
  if (requestedShift) {
    const requesterBusy = await db.shift.findFirst({
      where: { userId: requesterId, date: requestedShift.date, id: { not: offeredShift.id } },
    });
    if (requesterBusy) throw conflict(`Ya tienes un turno el ${requestedShift.date}`);
  }
}
```

El truco está en `id: { not: ... }`: el turno que cada uno *entrega* no cuenta como conflicto, porque va a dejar de ser suyo. Así, cambiar tu mañana del día 14 por la noche del compañero del mismo día 14 es válido.

El parámetro `db = prisma` permite reutilizar la función dentro de una transacción, pasándole `tx` en lugar de `prisma`.

Responder como compañero: `respondAsPeer`

- Solo puede responder el **destinatario**, y solo si el estado es `PENDING_PEER`.
- Si **rechaza**: estado `REJECTED`, se guarda quién y cuándo, y se avisa al solicitante.
- Si **acepta**: se vuelven a comprobar los conflictos (pudo pasar tiempo), el estado pasa a `PENDING_SUPERVISOR` y se avisa al solicitante y a **todos los supervisores** de la planta.

Aprobar como supervisor: `resolveAsSupervisor`

Aquí se modifican de verdad los turnos, y todo ocurre dentro de una **transacción**:

```
const updated = await prisma.$transaction(async (tx) => {
  // 1. Releer los turnos: ¿siguen siendo de quien eran?
  const offered = await tx.shift.findUnique({ where: { id: swap.offeredShiftId } });
  const requested = swap.requestedShiftId ? await tx.shift.findUnique(...) : null;
  if (offered.userId !== swap.requesterId || (requested && requested.userId !== swap.targetId))
{
    throw conflict('Los turnos han cambiado desde que se creó la solicitud');
  }
  // 2. Volver a comprobar conflictos
  await assertNoConflicts({ ...swap, offeredShift: offered, requestedShift: requested }, tx);

  // 3. Intercambiar los dueños (ver "el truco de la fecha temporal")
  if (requested) {
    await tx.shift.update({ where: { id: requested.id }, data: { date: `tmp-${requested.id}` }
});
    await tx.shift.update({ where: { id: offered.id },   data: { userId: swap.targetId } });
    await tx.shift.update({ where: { id: requested.id }, data: { userId: swap.requesterId, date:
requested.date } });
  } else {
    await tx.shift.update({ where: { id: offered.id }, data: { userId: swap.targetId } });   //
cesión
  }

  // 4. Cancelar otras solicitudes pendientes sobre estos turnos (ya no tienen sentido)
  await tx.swapRequest.updateMany({ where: { id: { not: swap.id }, status: { in:
PENDING_STATUSES },
    OR: [{ offeredShiftId: { in: shiftIds } }, { requestedShiftId: { in: shiftIds } }] },
    data: { status: 'CANCELLED', resolvedAt: new Date() } });

  // 5. Marcar como aprobada
  return tx.swapRequest.update({ where: { id: swapId }, data: { status: 'APPROVED', ... } });
});
```

¿Qué es una transacción y por qué hace falta aquí?

«Una transacción agrupa varias operaciones en la base de datos para que se apliquen todas o ninguna. Al aprobar un intercambio hay que cambiar el dueño de dos turnos y actualizar la solicitud. Si el servidor fallara a mitad, sin transacción podríamos quedarnos con los dos turnos asignados a la misma persona. Con `prisma.$transaction`, si cualquier paso lanza un error, se deshace todo (*rollback*).»

Esto corresponde a la **A de ACID**: *Atomicidad*.

¿Por qué volver a validar al aprobar?

Entre que Lucía crea la propuesta y la supervisora la aprueba pueden pasar días. En ese tiempo pueden haber cambiado cosas:

- La supervisora asignó a Marcos otro turno ese mismo día.
- Otro intercambio movió uno de los turnos.

Por eso se relee todo **dentro de la transacción** y se vuelve a comprobar. Es un principio general: **validar justo antes de actuar**, no solo al principio.

El truco de la fecha temporal

Problema: Lucía tiene `N` el día 14 y Marcos `M` el día 14. Quieren intercambiarlos. Hay que hacer dos `UPDATE`, uno detrás de otro:

1. Pasar el turno de Lucía a Marcos. **Error**: en ese momento Marcos tendría dos turnos el día 14, y la restricción `@@unique([userId, date])` lo impide.

Solución en tres pasos:

Paso	Turno de Lucía (N)	Turno de Marcos (M)
Inicio	Lucía · 14	Marcos · 14
1. Fecha temporal al de Marcos	Lucía · 14	Marcos · `tmp-57`
2. El de Lucía pasa a Marcos	**Marcos** · 14	Marcos · `tmp-57`
3. El de Marcos pasa a Lucía y recupera su fecha	Marcos · 14	**Lucía** · 14

En ningún momento hay dos turnos de la misma persona con la misma fecha. Como todo está dentro de la transacción, nadie llega a ver nunca la fecha `tmp-`. Hay un test específico para este caso.

Cancelar y listar

- `cancelSwap`: solo quien creó la solicitud, y solo si sigue pendiente. Se avisa al compañero.
- `listSwaps`: el supervisor ve todas las de su planta; un sanitario ve aquellas en las que es solicitante o destinatario (`OR`).

Interacción con los cambios del supervisor

Si el supervisor modifica un turno desde el cuadrante (`PATCH /shifts/:id`), se llama a `cancelPendingSwapsFor(id)`: cualquier intercambio pendiente sobre ese turno pasa a `CANCELLED`. Así no se aprueba un cambio sobre un turno que ya no es el que se acordó.

13. Notificaciones

Cómo se crean

```
export function notify(userId, message, link = null, db = prisma) {
  return db.notification.create({ data: { userId, message, link } });
}
```

Es una función mínima a la que se llama desde los servicios y las rutas cada vez que pasa algo relevante:

Evento	A quién se avisa
Supervisor crea, cambia o borra un turno	Al sanitario afectado (o a los dos, si cambia de persona)
Alguien te propone un intercambio	Al destinatario
El compañero acepta	Al solicitante y a los supervisores de la planta
El compañero rechaza	Al solicitante
El supervisor aprueba o deniega	A los dos implicados
El solicitante cancela	Al destinatario

Cómo llegan al usuario: *polling*

El componente `Layout` pide `/notifications` y `/swaps` **cada 30 segundos** y cada vez que cambias de página, y calcula los números rojos de las pestañas:

```
useEffect(() => {
  let alive = true;
  async function refresh() { /* pide avisos e intercambios, cuenta los pendientes */ }
  refresh();
  const id = setInterval(refresh, 30_000);
  return () => { alive = false; clearInterval(id); };   // limpieza al desmontar
}, [location.pathname, isSupervisor, user.id]);
```

- La función de limpieza de `useEffect` detiene el intervalo. Sin ella, se acumularían intervalos cada vez que cambias de página.
- La variable `alive` evita actualizar el estado si el componente ya no está en pantalla cuando llega la respuesta.

> **¿Por qué polling y no WebSockets?**
> Es lo más simple y suficiente para este caso: un aviso de turno no necesita llegar en milisegundos. Una mejora futura sería usar WebSockets o Server-Sent Events para recibir avisos en tiempo real, o notificaciones push al móvil.

14. El frontend: React por dentro

Arranque

frontend/src/main.jsx

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>          {/* habilita las rutas */}
      <AuthProvider>         {/* comparte el usuario logueado con toda la app */}
        <App />
      </AuthProvider>
    </BrowserRouter>
  </StrictMode>
);
```

Rutas y protección

frontend/src/App.jsx

```
function RequireAuth({ children }) {
  const { user, loading } = useAuth();
  if (loading) return <p>Cargando…</p>;
  return user ? children : <Navigate to="/login" replace />;
}

<Routes>
  <Route path="/login" element={<LoginPage />} />
  <Route element={<RequireAuth><Layout /></RequireAuth>}>   {/* ruta "layout" */}
    <Route index element={<CalendarPage />} />
    <Route path="cuadrante" element={<RosterPage />} />
    <Route path="intercambios" element={<SwapsPage />} />
    <Route path="notificaciones" element={<NotificationsPage />} />
  </Route>
  <Route path="*" element={<Navigate to="/" replace />} />
</Routes>
```

Las páginas están anidadas dentro de una ruta que pinta `Layout` (cabecera y pestañas). `Layout` tiene un `<Outlet />`, que es el hueco donde React Router coloca la página actual. Así la cabecera no se repite en cada página.

El contexto de autenticación

Problema: muchas partes de la app necesitan saber quién está logueado. Pasarlo de componente en componente con props sería muy incómodo (*prop drilling*). Solución: **React Context**.

frontend/src/lib/auth.jsx (resumido)

```
export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(Boolean(tokenStore.get()));

  useEffect(() => {                       // al cargar: si hay token, ¿sigue siendo válido?
    if (!tokenStore.get()) return;
    api.get('/auth/me').then(({ user }) => setUser(user))
       .catch(() => tokenStore.clear())
       .finally(() => setLoading(false));
  }, []);

  async function login(email, password) { /* POST /auth/login, guarda token, GET /auth/me */ }
  function logout() { tokenStore.clear(); setUser(null); }

  const isSupervisor = user?.role === 'SUPERVISOR';
  return <AuthContext.Provider value={{ user, loading, login, logout, isSupervisor }}>...
}
export const useAuth = () => useContext(AuthContext);
```

Cualquier componente hace `const { user, isSupervisor } = useAuth()` y listo. Como al recargar la página el estado de React se pierde, el token se guarda en `localStorage` y al arrancar se comprueba con `/auth/me`.

El cliente de la API

`frontend/src/lib/api.js (resumido)`

```
async function request(method, path, body) {
  const token = tokenStore.get();
  const res = await fetch(`/api${path}`, {
    method,
    headers: {
      ...(body && { 'Content-Type': 'application/json' }),
      ...(token && { Authorization: `Bearer ${token}` }),
    },
    body: body ? JSON.stringify(body) : undefined,
  });
  if (res.status === 401 && token) { tokenStore.clear(); window.location.href = '/login'; }
  if (res.status === 204) return null;
  const data = await res.json().catch(() => null);
  if (!res.ok) throw new ApiError(res.status, data?.error ?? 'Error de conexión con el
servidor');
  return data;
}
export const api = { get, post, patch, delete };
```

Centraliza en un único sitio:

- añadir el token a cada petición;
- convertir el cuerpo a JSON;
- si la sesión ha caducado (401), volver al login;
- convertir las respuestas de error en excepciones con el mensaje del backend, que los componentes muestran con `err.message`.

Detalle: `fetch` **no** lanza error con un 404 o un 500, solo con fallos de red. Por eso hay que comprobar `res.ok` a mano.

El hook `useFetch`

```
export function useFetch(path) {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(true);

  const reload = useCallback(() => {
    if (!path) return Promise.resolve();
    setLoading(true);
    return api.get(path).then((d) => { setData(d); setError(null); })
      .catch(setError).finally(() => setLoading(false));
  }, [path]);

  useEffect(() => { reload(); }, [reload]);
  return { data, error, loading, reload };
}
```

- Un **custom hook** encapsula la lógica repetida de «cargar datos al montar».
- Cuando cambia `path` (por ejemplo, al cambiar de mes), `useCallback` crea una nueva función `reload`, el `useEffect` se vuelve a ejecutar y se recargan los datos solos.
- `reload` se devuelve para refrescar después de una acción (aceptar, guardar…).
- Si `path` es `null`, no se carga nada. Se usa en el modal de intercambio, que no pide los turnos hasta que eliges compañero.

Utilidades de fechas

`lib/dates.js` trabaja siempre con textos `"YYYY-MM-DD"` en hora **local**. Funciones destacadas:

- `toISO(date)` / `fromISO(str)`: convierten entre `Date` y texto. No se usa `toISOString()` porque devuelve la fecha en UTC, y de madrugada daría el día anterior.
- `monthWeeks(year, month)`: genera las semanas del mes empezando en **lunes** (en España la semana empieza en lunes; `getDay()` devuelve 0 para el domingo, de ahí el `(getDay() + 6) % 7`) y rellena los huecos con `null`.
- `new Date(año, mes + 1, 0)`: el «día 0» del mes siguiente es el último día del mes actual. Es un truco clásico para saber cuántos días tiene un mes.
- `toLocaleDateString('es-ES', ...)`: nombres de días y meses en español sin librerías.

15. Las páginas y componentes

Componentes reutilizables

Componente	Qué hace	Detalles interesantes
`Layout`	Cabecera, pestañas y badges	Usa `NavLink`, que añade la clase `active` a la pestaña actual. Hace el polling de avisos.
`Modal`	Ventana emergente genérica	Se cierra con la tecla Escape (escucha `keydown` y limpia el listener) y al pulsar fuera. Tiene `role="dialog"` por accesibilidad.
`MonthNav`	Flechas ‹ › y botón «Hoy»	`new Date(year, month + delta, 1)` gestiona solo el cambio de año (de diciembre a enero).
`ShiftChip`	Etiqueta de color M/T/N	Muestra el horario en el `title` (tooltip).
`SwapModal`	Formulario para proponer un intercambio	Ver abajo.

`SwapModal`: lógica en el frontend para guiar al usuario

El backend valida todo, pero el frontend **ayuda** a no equivocarse:

```
const { theirShifts, busyOnMyDay } = useMemo(() => {
  const all = unitShifts ?? [];
  // días en los que YO trabajo (sin contar el turno que ofrezco)
  const myDates = new Set(all.filter((s) => s.userId === user.id && s.id !== shift.id).map((s)
=> s.date));
  return {
    // turnos del compañero en días en los que yo estoy libre
    theirShifts: all.filter((s) => s.userId === Number(targetId) && !myDates.has(s.date)),
    // ¿el compañero ya trabaja el día que le ofrezco?
    busyOnMyDay: all.find((s) => s.userId === Number(targetId) && s.date === shift.date),
  };
}, [unitShifts, targetId, user.id, shift]);
```

- `useMemo` evita recalcular estos filtros en cada render; solo se recalculan si cambian sus dependencias.
- `Set` permite comprobar «¿trabajo este día?» de forma instantánea con `has()`.
- Si el compañero ya trabaja ese día, solo se ofrece cambiar el turno de ese mismo día y se desactiva la cesión, con un mensaje explicativo.
- El botón «Enviar» se desactiva (`canSubmit`) hasta que el formulario es coherente.

> **Validar en frontend Y en backend**
>
> La validación del frontend es por **comodidad** (avisa antes y evita errores). La del backend es por **seguridad** (es la que manda). Siempre hacen falta las dos.

Páginas

CalendarPage («Mis turnos»)

- Hace tres peticiones: los turnos del mes que se ve, los de los próximos 30 días (para la tarjeta «próximo turno», que no depende del mes visible) y los intercambios (para el punto rojo).
- Convierte la lista de turnos en un objeto indexado por fecha (`Object.fromEntries`), para saber al instante qué turno hay cada día: `byDate['2026-10-14']` .
- Calcula los totales del mes con `reduce` .
- Las clases CSS de cada celda se construyen con un array y `.filter(Boolean).join(' ')` : una forma limpia de añadir clases condicionales (`today` , `past` , `selected` …).

RosterPage («Cuadrante»)

- Una tabla HTML con una fila por persona y una columna por día.
- Los turnos se indexan en un objeto con la clave `"userId|fecha"` para encontrar cada celda rápido.
- La primera columna es `position: sticky` : al desplazarse en horizontal, los nombres se quedan fijos.
- Si eres supervisor, cada celda es un botón que abre `ShiftEditor` . Este decide si hay que hacer `POST` (no había turno), `PATCH` (cambiar) o `DELETE` (elegir «Libre»).
- Incluye `AddStaffModal` para dar de alta personal.

SwapsPage («Intercambios»)

- Agrupa las solicitudes en secciones distintas según el rol (las que requieren tu acción, en curso e historial).
- `actionsFor(swap)` decide qué botones mostrar según quién eres y el estado. Es el reflejo en la interfaz de las reglas del backend.
- Mientras se procesa una acción, los botones de esa tarjeta se desactivan (`busy`) para evitar dobles clics.

NotificationsPage («Avisos») y LoginPage

- Avisos: lista con los no leídos resaltados; al pulsar uno se marca como leído y se navega a su `link` .
- Login: si ya hay usuario, redirige a `/` con `<Navigate>` . Incluye los botones de demo.

Patrones de React que aparecen en el proyecto

Concepto	Dónde
`useState`	En todos los componentes (formularios, mes visible, modal abierto…)
`useEffect` + limpieza	Layout (intervalo), Modal (listener de teclado), AuthProvider
`useMemo`	SwapModal
`useCallback`	useFetch
`useContext`	useAuth

Custom hooks	useFetch, useAuth
Componentes controlados	Todos los inputs (su valor viene del estado)
Renderizado condicional	`{flash && <p>...}`, ternarios
Listas con `key`	Todas las listas usan el `id` como key
Elevar el estado	La página tiene el estado del modal y le pasa `onClose` y `onDone`

16. Estilos y diseño responsive

Todos los estilos están en `frontend/src/styles.css`, sin librerías.

Variables CSS (design tokens)

```
:root {
  --primary: #0f766e;     /* verde azulado, color "sanitario" */
  --m-bg: #fef3c7; --m-fg: #92400e;   /* mañana: ámbar */
  --t-bg: #e0f2fe; --t-fg: #075985;   /* tarde: azul cielo */
  --n-bg: #e0e7ff; --n-fg: #3730a3;   /* noche: índigo */
}
```

Cambiar un color en un sitio lo cambia en toda la app. Los colores de turno siguen una lógica intuitiva: cálido para la mañana y oscuro para la noche.

Técnicas usadas

- **CSS Grid** para el calendario: `grid-template-columns: repeat(7, 1fr)`, siete columnas iguales.
- **Flexbox** para cabeceras, botones y tarjetas.
- `position: sticky` para la columna de nombres del cuadrante.
- `overflow-x: auto` en el cuadrante: en el móvil se desplaza en horizontal sin romper la página.
- **Media query** `@media (max-width: 640px)`: en el móvil, las pestañas se reparten el ancho con el icono arriba, el calendario se compacta y las tarjetas de intercambio pasan a una columna.
- `::first-letter` para poner en mayúscula la primera letra de «octubre de 2026». `text-transform: capitalize` no sirve, porque pondría «Octubre De 2026».

Accesibilidad básica

- Los elementos pulsables son `<button>` de verdad, no `<div>`, así que funcionan con el teclado.
- Hay un contorno visible al navegar con el tabulador (`:focus-visible`).
- Los botones que solo tienen un icono llevan `aria-label` (por ejemplo, «Mes anterior»).
- Los modales tienen `role="dialog"` y se cierran con Escape.

¿Por qué «mobile first»?

El personal sanitario no está delante de un ordenador: consulta su turno desde el móvil, en el autobús o en el descanso. Por eso cada pantalla se diseñó para funcionar a 390 px de ancho.

17. Datos de demo (seed)

`backend/prisma/seed.js` rellena la base de datos con datos realistas:

- 1 planta: «Planta 3 · Medicina Interna».
- 1 supervisora (Elena Navarro) y 6 sanitarios: 4 de enfermería y 2 TCAE.
- Contraseña `turnos123` para todos (cifrada con bcrypt).
- Turnos desde el día 1 del mes actual hasta el final del mes siguiente.

La rotación

```
// Rotación clásica de planta: 2 mañanas, 2 tardes, 2 noches, 2 libres
const ROTATION = ['M', 'M', 'T', 'T', 'N', 'N', null, null];

staff.forEach((user, i) => {
  // Desfase de 3 días entre personas para que cada turno quede cubierto
  const type = ROTATION[(day + i * 3) % ROTATION.length];
  if (type) shifts.push({ date: fmt(d), type, userId: user.id, unitId: unit.id });
});
```

El operador `%` (módulo) hace que la rotación se repita cada 8 días. El desfase `i * 3` hace que cada persona esté en un punto distinto del ciclo, así que cada día hay gente en todos los turnos (se ve en la fila de cobertura). Al final se insertan todos los turnos de golpe con `createMany`, mucho más rápido que uno a uno.

El seed borra primero todos los datos, en el orden inverso a las dependencias (primero avisos y solicitudes, al final plantas), para no romper las claves foráneas.

18. Tests

Son **tests de integración**: lanzan peticiones HTTP reales contra la app de Express y comprueban las respuestas y la base de datos. No se simula (*mock*) nada: se prueba el sistema completo (rutas, middleware, servicio, Prisma y SQLite).

Infraestructura

- **`vitest.config.js`**: usa una base de datos aparte (`test.db`) para no tocar la de desarrollo, define un `JWT_SECRET` de prueba y ejecuta los archivos **en serie** (`fileParallelism: false`), porque todos comparten la misma base de datos.
- **`globalSetup.js`**: antes de todos los tests, ejecuta `prisma db push` para crear las tablas en `test.db`.
- **`helpers.js`**:
  - `resetDb()`: vacía las tablas. Se llama en `beforeEach`, así cada test empieza limpio y no depende de los demás.
  - `createWorld()`: crea un «mundo» de prueba: una planta, una supervisora (Sara), dos sanitarios (Ana y Luis) y alguien de otra planta (Pepe).
  - `dayOffset(n)`: la fecha de dentro de *n* días. Los tests usan fechas relativas a hoy para que sigan funcionando en el futuro.
  - `as(user)`: un cliente HTTP que ya lleva el token de ese usuario. Hace los tests muy legibles: `as(w.ana).post('/api/swaps', {...})`.

Qué se prueba (19 tests)

Archivo	Test
auth.test.js	Login correcto, sin exponer el hash
	Contraseña incorrecta → 401
	Rutas protegidas sin token → 401
	Un sanitario no puede crear turnos → 403
	El supervisor crea un turno y se notifica al sanitario
	Dos turnos el mismo día → 409
	No se asignan turnos a personal de otra planta → 400

swaps.test.js	Flujo completo: proponer → aceptar → aprobar; los turnos cambian de dueño
	Intercambio del mismo día (mañana por noche): prueba el truco de la fecha temporal
	Cesión de un turno sin recibir otro
	El supervisor no puede aprobar antes de que acepte el compañero → 409
	El rechazo del compañero no toca los turnos
	No se puede ofrecer un turno ajeno → 403
	No se pueden intercambiar turnos pasados → 400
	No se puede intercambiar con alguien de otra planta → 400
	Detecta que el compañero ya trabaja ese día → 409
	No permite dos solicitudes pendientes sobre el mismo turno → 409
	Si el supervisor modifica el turno, el intercambio pendiente se cancela
	Un sanitario solo ve sus solicitudes; el supervisor ve las de la planta

Ejemplo de test

```
it('intercambia turnos del mismo día (mañana por noche)', async () => {
  const date = dayOffset(2);
  const a = await shift(w.ana, date, 'M');
  const l = await shift(w.luis, date, 'N');

  await fullFlow({ offeredShiftId: a.id, targetId: w.luis.id, requestedShiftId: l.id });

  const ana = await prisma.shift.findFirst({ where: { userId: w.ana.id, date } });
  const luis = await prisma.shift.findFirst({ where: { userId: w.luis.id, date } });
  expect(ana.type).toBe('N');
  expect(luis.type).toBe('M');
});
```

Sigue el patrón **Arrange · Act · Assert**: preparar los datos, ejecutar la acción y comprobar el resultado.

> **¿Por qué testear sobre todo los permisos y los casos que deben fallar?**
>
> Porque es donde se esconden los bugs graves. Que el caso feliz funcione se ve usando la app; que un sanitario *no pueda* aprobar su propio intercambio solo se garantiza con tests.

19. Cómo arrancarlo y comandos útiles

Primera vez

```
npm run install:all    # instala dependencias de backend y frontend
npm run setup          # crea backend/.env, la base de datos y los datos de demo
```

Cada vez que quieras usarlo (dos terminales)

```
npm run dev:backend    # API en http://localhost:3000 (se reinicia sola al guardar)
npm run dev:frontend   # web en http://localhost:5173 (recarga en caliente)
```

Abre **http://localhost:5173** y usa los botones de demo. Para parar, pulsa `Ctrl+C` en cada terminal.

Todos los comandos

Comando	Dónde	Qué hace
`npm test`	raíz o backend	Ejecuta los 19 tests
`npm run db:seed`	backend	Vuelve a los datos de demo iniciales
`npm run db:push`	backend	Aplica cambios del `schema.prisma` a la base de datos
`npm run db:reset`	backend	Borra la base de datos, la recrea y carga el seed
`npx prisma studio`	backend	Abre una web para ver y editar la base de datos (muy útil para depurar)
`npm run build`	frontend	Genera la versión de producción en `dist/`

`node --watch` (en `npm run dev` del backend) reinicia el servidor al guardar un archivo. Viene integrado en Node, sin necesidad de nodemon.

Si cambias el modelo de datos

1. Edita `backend/prisma/schema.prisma`.
2. Ejecuta `npm run db:push` en `backend/`: actualiza la base de datos y regenera el cliente de Prisma.
3. Si hace falta, actualiza `seed.js` y ejecuta `npm run db:seed`.

20. Limitaciones conocidas y mejoras futuras

Saber qué le falta a tu proyecto demuestra criterio. En una entrevista es mejor reconocerlo tú antes de que te lo señalen.

Limitaciones actuales

Limitación	Por qué importa	Cómo se mejoraría
El token se guarda en `localStorage`	Si hubiera una vulnerabilidad XSS, un script malicioso podría leerlo	Guardarlo en una cookie `httpOnly` + `SameSite`, que JavaScript no puede leer
Sin límite de intentos de login	Permite ataques de fuerza bruta	`express-rate-limit` en `/auth/login`
Sin *refresh tokens*	Tras 7 días hay que volver a entrar; un token robado vale 7 días	Token de acceso corto (15 min) + refresh token
Polling cada 30 s	Los avisos no son instantáneos y se hacen peticiones aunque no haya cambios	WebSockets o Server-Sent Events
«Hoy» se calcula con la hora del servidor	Si el servidor está en otra zona horaria, podría desfasarse cerca de medianoche	Fijar la zona horaria (`TZ=Europe/Madrid`) o guardarla por planta
SQLite	No es adecuada para muchos usuarios escribiendo a la vez	PostgreSQL (cambiar `provider` en Prisma)
Sin tests de frontend	Un cambio en la interfaz podría romper algo sin que nos enteremos	Vitest + React Testing Library, o Playwright para pruebas de punta a punta
El usuario no puede cambiar su contraseña	La contraseña inicial la pone el supervisor	Endpoint de cambio de contraseña y enlace de restablecimiento por email
Sin reglas de descanso	Se puede encadenar una noche con una mañana	Validar un descanso mínimo entre turnos
Sin paginación en intercambios	Con años de historial, la lista crecería mucho	Paginación o filtro por fechas

Funcionalidades para el futuro

- Despliegue público (por ejemplo, Render o Railway para la API, Neon para PostgreSQL y la web en el mismo servidor o en Vercel).
- Notificaciones push o por email.
- Solicitudes de vacaciones y días libres.
- **Bolsa de turnos**: publicar un turno para que lo coja cualquier compañero.
- Exportar el cuadrante a PDF o al calendario del móvil (formato iCal).
- Plantillas de rotación para que el supervisor genere el cuadrante del mes automáticamente.

- Migración a TypeScript.
- Instalarla como app en el móvil (PWA).

21. Preguntas típicas de entrevista

Practica estas respuestas en voz alta. No hace falta aprenderlas de memoria: lo importante es entender la idea.

¿Qué problema resuelve tu proyecto?

En las plantas de hospital los turnos y sus cambios se gestionan con papel y WhatsApp, y se pierde información. Mi app centraliza el cuadrante y formaliza los intercambios con un flujo de aprobación y avisos automáticos.

¿Cuál fue la parte más difícil?

Los intercambios. Hay muchas reglas: de quién es cada turno, conflictos de días, estados... Además, al aprobar hay que cambiar dos turnos de forma atómica. El caso más curioso fue intercambiar dos turnos del mismo día, que chocaba con la restricción única de la base de datos; lo resolví con una fecha temporal dentro de la transacción.

¿Cómo funciona la autenticación?

Con JWT. En el login compruebo la contraseña con bcrypt y, si es correcta, firmo un token con el id del usuario y una clave secreta. El frontend lo guarda y lo envía en la cabecera `Authorization`. Un middleware lo verifica en cada petición y carga el usuario en `req.user`.

¿Por qué no guardas las contraseñas tal cual?

Porque si roban la base de datos, tendrían todas las contraseñas, y mucha gente las reutiliza. Guardo un hash de bcrypt, que no se puede revertir, lleva sal y es lento a propósito para dificultar la fuerza bruta.

¿Qué diferencia hay entre 401 y 403?

401: no sé quién eres (no hay token o no es válido). 403: sé quién eres, pero no tienes permiso. Por ejemplo, un sanitario que intenta crear un turno recibe un 403.

¿Qué es una transacción y dónde la usas?

Un grupo de operaciones que se aplican todas o ninguna. La uso al aprobar un intercambio: cambio el dueño de dos turnos, cancelo otras solicitudes y marco la solicitud como aprobada. Si algo falla a mitad, se deshace todo.

¿Cómo evitas que alguien tenga dos turnos el mismo día?

En dos niveles. En el código compruebo los conflictos antes de crear o aprobar un intercambio. Y en la base de datos hay una restricción única sobre `(userId, date)`, que es la última garantía aunque el código fallara.

¿Qué pasa si un sanitario intenta aprobar un intercambio llamando a la API directamente?

Recibe un 403. Ocultar el botón en el frontend es solo comodidad; los permisos se comprueban siempre en el backend, con el middleware `requireSupervisor`. Hay tests que lo verifican.

¿Por qué separaste rutas y servicios?

Las rutas se encargan de HTTP (leer la petición, validar y responder) y el servicio de las reglas de negocio. Así el código está mejor organizado y la lógica compleja queda en un solo sitio, fácil de leer y de testear.

¿Cómo validas los datos de entrada?

Con Zod. Cada ruta tiene un esquema; si los datos no cumplen, se lanza un error que el manejador global convierte en un 400. Además, Zod descarta campos no esperados, lo que evita que alguien se cuele un campo como `role`.

¿Cómo gestionas el estado global en React?

Con Context para el usuario logueado (`AuthProvider` y el hook `useAuth`). Para los datos de cada página uso un hook propio, `useFetch`, que carga los datos y expone `reload`. No necesité Redux porque el estado global es muy pequeño.

¿Cómo testeaste la app?

Con tests de integración usando Vitest y Supertest: lanzo peticiones reales a la API contra una base de datos de test que se limpia antes de cada test. Me centré en los permisos y en las reglas de los intercambios, sobre todo en los casos que deben fallar.

¿Por qué guardas la fecha como texto?

Porque un turno es de un día, no de un instante. Con `DateTime` la zona horaria podría mover el turno al día anterior. El formato `YYYY-MM-DD` evita eso y además se ordena y compara correctamente como texto.

¿Qué mejorarías?

Pasaría el token a una cookie `httpOnly`, añadiría un límite de intentos en el login, cambiaría a PostgreSQL para desplegar, sustituiría el polling por WebSockets y añadiría tests de frontend. Como funcionalidades: vacaciones, una bolsa de turnos y notificaciones push.

¿Usaste IA para hacerlo?

Responde con sinceridad. Es habitual y está bien visto si lo cuentas con naturalidad y demuestras que entiendes cada parte. Por ejemplo: «Me apoyé en herramientas de IA para avanzar más rápido, pero revisé y entiendo todo el código; puedo explicarte cualquier parte». Para eso sirve este documento.

¿Cómo escalaría si lo usara un hospital entero?

PostgreSQL en lugar de SQLite; el modelo ya separa por plantas (`Unit`), así que admite muchas. Añadiría paginación, notificaciones en tiempo real y, si hiciera falta, varias instancias de la API detrás de un balanceador. Como la autenticación es con JWT y no hay sesiones en memoria, la API ya puede escalar horizontalmente.

22. Glosario

Término	Significado
API REST	Forma de diseñar un backend en la que los recursos (turnos, usuarios…) se manipulan con URLs y métodos HTTP (GET, POST, PATCH, DELETE).
SPA	Single Page Application: la web carga una vez y luego cambia de «página» con JavaScript, sin recargar.
ORM	Object-Relational Mapping: permite trabajar con la base de datos usando objetos y funciones en lugar de SQL.
Middleware	Función que se ejecuta entre la llegada de la petición y la ruta final (autenticación, parseo de JSON…).
JWT	JSON Web Token: token firmado que identifica al usuario sin guardar sesiones en el servidor.
Hash	Transformación de un solo sentido. Se usa para guardar contraseñas de forma segura.
Sal (salt)	Valor aleatorio que se añade antes de calcular el hash para que dos contraseñas iguales den hashes distintos.
Transacción	Grupo de operaciones que se ejecutan todas o ninguna.
ACID	Atomicidad, Consistencia, Aislamiento y Durabilidad: las garantías de una base de datos transaccional.
Restricción única	Regla de la base de datos que impide que dos filas tengan los mismos valores en ciertas columnas.
Clave foránea	Columna que apunta a una fila de otra tabla (`userId` → User).
Índice	Estructura que acelera las búsquedas por ciertas columnas.
Máquina de estados	Modelo en el que algo solo puede estar en ciertos estados y pasar de uno a otro por transiciones permitidas.
CORS	Mecanismo del navegador que bloquea peticiones a otro origen salvo que el servidor lo permita.
Proxy (Vite)	Reenvío de las peticiones `/api` del frontend al backend durante el desarrollo.
Seed	Script que rellena la base de datos con datos de ejemplo.
Test de integración	Test que prueba varias piezas juntas (API + base de datos), no una función aislada.
Polling	Preguntar al servidor cada cierto tiempo si hay novedades.
Hook	Función de React que empieza por `use` y permite usar estado y efectos en componentes.
Context	Mecanismo de React para compartir datos con muchos componentes sin pasarlos por props.

Responsive	Diseño que se adapta a distintos tamaños de pantalla.
Mass assignment	Vulnerabilidad en la que el cliente envía campos que no debería poder modificar (por ejemplo, su rol).
XSS	Cross-Site Scripting: inyectar JavaScript malicioso en una web para que se ejecute en el navegador de otros.